

初赛模拟卷14

一、单项选择题（共15题，每题2分，共计30分;每题有且仅有一个正确选项）

1. 以下哪位科学家被称为“博弈论之父”，“现代计算机之父”? ()
A.冯诺依曼
B.塔扬
C.图灵
D.比尔盖茨
2. 下列属于网络模型的名称是 ()。
A.FTP
B.SMTP
C.TCP/IP
D.LAN
3. 设有 100 个顶点，利用二分法查找时，最大比较次数是()。
A.25
B.10
C.7
D.50
4. 学号为 1 到 30 的小朋友顺时针排成一圈，从1号小朋友开始顺时针报数，从数字 1 开始数下去，1，2，3，...，28，29，30，31，32，一圈又一圈，问当数到数字 n ，所在的小朋友的学号为多少? ()。
A. $(n + 1) \% 30 - 1$
B. $(n - 1) \% 30$
C. $1 + (n - 1) \% 30$
D. $(n + 1) \% 30$

5. 十进制小数 13.375 对应的二进制数是()。
- A.1101.101
 - B.1011.011
 - C.1010.01
 - D.1101.011
6. 计算机中的数有浮点与定点两种，其中用浮点表示的数，通常由()这两部分组成。
- A.尾数与小数
 - B.阶码与尾数
 - C.指数与基数
 - D.整数与小数
7. 表达式 $(1+34) * 5 - 56 / 7$ 的后缀表达式为 ()
- A. 1 34 + 5 * 56 7 / -
 - B. 1 34 + 5 56 7 - * /
 - C. - * + 1 34 5 / 56 7
 - D. 1 34 5 * + 56 7 /
8. 8 位二进制数中去掉符号位，最大能表示多少字符 ()
- A.128
 - B.127
 - C.255
 - D.256
9. 已知一棵二叉树前序遍历为 ABCDEFGI ,后序遍历为 CEDBIGFA , 则其中序遍历可能为 ()
- A. CBDEAGFI
 - B. ABCDEFGI
 - C. CBEDAIFG
 - D. CBEDAFIG
10. 在无向图中，所有顶点的度数之和是边数的 () 倍。
- A.0.5
 - B.1
 - C.2
 - D.4
11. 8 颗子弹，编号为 1、2、3、4、5、6、7、8，从编号 1 开始按序嵌入弹夹，以下有哪个不是正常的打出子弹的次序 ()
- A. 87654321
 - B. 32154876
 - C. 32164587
 - D. 12345678
12. 有 A、B、C、D、E、F 六个绝顶聪明又势均力敌的盗墓贼，他们都排着队，他们每个人都想独吞财宝，最前面的 A 如果拿了财宝，那么体力下降，则其后面的 B 会杀掉 A，拿了财宝，当然 B 拿了财宝，体力也会下降，一样会被 C 杀掉，如果 B 不拿财宝，则 C

无法杀 B，请问 A、C、E 的最终想法是 ()

A.A不拿C不拿E不拿

B.A拿C拿E不拿

C.A不拿C拿E拿

D.A不拿C不拿E拿

13. 完全二叉树共有 $2 * N - 1$ 个结点，则它的叶节点数是 ()。

A. N

B. $N - 1$

C. $2 * N$

D. $2 * N - 1$

14. 如果用一个字节来表示整数，最高位用作符号位，其他位表示数值。例如：00000001 表示，10000001 表示 -1，试问这样表示法的整数 A 的范围应该是 ()。

A. $-128 < A < 128$

B. $-128 \leq A < 128$

C. $-128 \leq A \leq 127$

D. $-128 \leq A \leq 128$

15. 给出 3 种排序:插入排序、冒泡排序、选择排序。这 3 种排序的时间代价分别是 ()。

A. $O(n^2)$ 、 $O(n)$ 、 $O(n)$

A. $O(\log n)$ 、 $O(n)$ 、 $O(n^2)$

A. $O(n^2)$ 、 $O(n^2)$ 、 $O(n^2)$

A. $O(n)$ 、 $O(n^2)$ 、 $O(\log(n))$

二、阅读程序（程序输入不超过数组或字符串定义的范围;其中判断题1.5分，第5题分别为3,3,4分，第6题分别为4,4,4分，共计40分）

第一题

```
#include<iostream>
#include<cstdio>
using namespace std;
int i, j, n;
int x[101], y[101];
int main() {
    cin >> n;
    for (i = 1; i <= n; i++) cin >> x[i];
    for (i = 1; i < n; i++)
        for (j = i + 1; j <= n; j++)
```

```

        if (x[i] > x[j])
            y[j]++;
        else if (x[i] < x[j])
            y[i]++;
    for (i = 1; i <= n; i++)
        printf("%d ", y[i]);
    cout << endl;
    return 0;
}

```

1. 把第 12 行与第 14 行互换位置，结果不会改变。()
 - A.正确
 - B.错误
2. 第 13 行把 `if(x[i] < x[j])` 删掉效果一样。()
 - A.正确
 - B.错误
3. 数组 `y[i]` 中存的是 `x[i]` 在数列中从大到小的次序。()
 - A.正确
 - B.错误
4. 第 10 行把 `i + 1` 改成 `1`，数组 `y` 每个元素的值增加 1 倍。()
 - A.正确
 - B.错误
5. 此程序如果 `n` 输入 4，然后输入 2 4 1 3，输出结果是()。
 - A. 4 3 2 1
 - B. 1 2 3 4
 - C. 2 0 3 1
 - D. 1 3 0 2
6. 此程序的时间复杂度是()
 - A. $O(\log n)$
 - B. $O(n^2)$
 - C. $O(n \log n)$
 - D. $O(n)$

第二题

```

#include<iostream>
#include<cstdio>
using namespace std;
int j, i, m;
int a[10];

```

```

int main() {
    for (i = 2; i <= 6; i++)
        a[i] = i + 1;
    do {
        m = 2;
        for (i = 3; i <= 6; i++)
            if (a[m] > a[i]) m = i;
            a[m] = a[m] + m;
        m = 1;
        for (i = 2; i <= 5; i++)
            for (j = i + 1; j <= 6; j++)
                if (a[i] < a[j]) m = 0;
    } while (m == 0);
    printf("%d", a[2]);
    return 0;
}

```

1. 程序结束时，a[2] 的值一定是数组 a 中的最大值。()
 - A.正确
 - B.错误
2. 第 18 行 m==0 成立时，数组 a[i] ($2 \leq i \leq 6$) 从大到小排序:()
 - A.正确
 - B.错误
3. 程序输出时，a 数组满足:对任意的 $2 \leq i < 6$ ，有 $a[i] > a[i+1]$ 。()
 - A.正确
 - B.错误
4. 删除第 14 行代码 m=1 程序结果会发生改变。()
 - A.正确
 - B.错误
5. 程序的输出结果为()
 - A.58
 - B.61
 - C.59
 - D.60
6. 第 17 行 if (a[i] < a[j]) 执行了多少次()。
 - A. 800
 - B. 820
 - C. 10
 - D. 30

第三题

```

#include<bits/stdc++.h>
using namespace std;
int a[6];
int change(int a) {
    a++;
}
int change1(int & a) {
    a++;
}
int main() {
    int c = 1;
    for (int i = 1; i <= 5; i++) a[i] = i * 3;
    int * b = & a[1];
    change( * b);
    cout << * b << endl;
    cout << a[1] << endl;
    * b++;
    cout << * b << endl;
    cout << a[1] << endl;
    change1( * b);
    cout << * b << endl;
    cout << a[1] << endl;
    * b = c;
    change(c);
    cout << * b << endl;
    cout << c << endl;
    change1(c);
    cout << * b << endl;
    cout << c << endl;
    return 0;
}

```

1. 将第 11 行中 int 换为 long long 后程序依然能通过编译。()
 - A.正确
 - B.错误
2. change 与 change1 两个函数等价。()
 - A.正确
 - B.错误
3. 将第 23 行换为 b = &c; 输出值不变。()
 - A.正确
 - B.错误
4. 将第 13 行 int * b = & a[1]; 换为 int *b=a+1; 输出值不变。()
 - A.正确

B.错误

5. 输出结果的最大值是。()

A.6

B.4

C.7

D.5

6. 输出结果的乘积是。()

A.13608

B.11520

C.5760

D.6804

三、完善程序（单选题，每题 3 分，共计 30 分）

第一题

小 C 到商店购物，她只带了 SW 元钱。有 n 件商品，第 i 件商品的价格为 w_i 元，小 C 对它的满意度为 v_i 。小 C 想要知道，用自己仅有的 SW 元钱，能买到的所有商品的满意度之和最大是多少。

数据保证 $1 \leq n, w_i, v_i \leq 100, 1 \leq SW \leq 10000$

```
#include <iostream>
using namespace std;

const int INF = 0x3f3f3f3f;

int n, SW, W[105], V[105], Dp[___(1)___];

int main(){
    cin >> n >> SW;
    for (int i = 1; i <= n; i++)
        cin >> W[i] >> V[i];
    for (int i = 1; i <= SW; i++)
        Dp[i] = ___(2)___;
    for (int i = 1; i <= n; i++)
        for (___(3)___)
            Dp[j] = max(Dp[j], ___(4)___);
    int ans = 0;
    for (int i = 1; i <= SW; i++)
        ___(5)___;
```

```

    cout << ans << "\n";
    return 0;
}

```

1. (1) 处应填 ()

- A. 100
- B. 105
- C. 10000
- D. 10005

2. (2) 处应填 ()

- A. INF
- B. 0
- C. -1
- D. 1

3. (3) 处应填 ()

- A. `int j = SW; j >= W[i]; j--`
- B. `int j = W[i]; j <= SW; j++`
- C. `int j = 1; j <= n; j++`
- D. `int j = n; j >= 1; j--`

4. (4) 处应填 ()

- A. `Dp[i - W[j]] + V[j]`
- B. `Dp[i - V[j]] + W[j]`
- C. `Dp[j - W[i]] + V[i]`
- D. `Dp[j - V[i]] + W[i]`

5. (5) 处应填 ()

- A. `ans = max(ans, Dp[i])`
- B. `ans = min(ans, Dp[i])`
- C. `ans = ans + Dp[i]`
- D. `ans = ans - Dp[i]`

第二题

(链表反转) 单向链表反转是一道经典算法问题，比如有一个链表是这样的 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ ，反转后成为 $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ 。现给定如下链表节点的定义：

```

struct LinkNode {
    int value;
    LinkNode* next;
};

```


非递归实现:

```
LinkNode* Reverse(LinkNode* header) {
    if (header == NULL || header->next == NULL) {
        return header;
    }
    LinkNode *pre = header, *cur = header->next;
    pre->next = NULL;
    while (cur != NULL) {
        LinkNode* next = ___(1)___;
        ___(2)___ = pre;
        pre = cur;
        cur = next;
    }
    return pre;
}
```

递归实现:

```
LinkNode* Reverse(LinkNode* head) {
    if (head == NULL || head->next == NULL) {
        return head;
    }
    LinkNode* newhead = ___(3)___;
    ___(4)___ = head;
    head->next = ___(5)___;
    return newhead;
}
```

1. (1) 中应填

- A. pre-> next
- B. cur-> next
- C. header-> next
- D. NULL

2. (2) 中应填

- A. pre-> next
- B. cur-> next
- C. header-> next
- D. NULL

3. (3) 中应填

- A. ReverseList(head)
- B. ReverseList(pre)

C. ReverseList(cur)

D. ReverseList(head -> next)

4. (4) 中应填

A. pre -> next -> next

B. cur -> next -> next

C. head -> next -> next

D. NULL

5. (5) 中应填

A. pre-> next

B. cur-> next

C. head-> next

D. NULL